Lab 0: RV64 内核调试

1 实验目的

安装虚拟机及Docker，通过在QEMU模拟器上运行Linux来熟悉如何从源代码开始将内核运行在QEMU模拟器上，学习使用GDB跟QEMU对代码进行联合调试，为后续实验打下基础。

2 实验内容及要求

- 安装虚拟机软件、Ubuntu镜像，自行学习Linux基础命令。
- 安装Docker，下载并导入Docker镜像，创建并运行容器。
- 编译内核并用 gdb + QEMU 调试，在内核初始化过程中设置断点，对内核的启动过程进行跟踪，并尝试使用gdb的各项命令。

请各位同学独立完成实验，任何抄袭行为都将使本次实验判为0分。请查看文档尾部附录部分的背景知识介绍，**跟随实验步骤完成实验并以截图的方式记录实验过程**，如有需要请对每一步的命令以及结果进行必要的解释。

最终提交的实验报告中**请删除附录部分**，并命名为"**学号_姓名_lab0.pdf**"，以pdf格式上传至学在浙大平台。

3 操作方法和实验步骤

3.1 通过虚拟机安装Linux系统(20%)

请参考【附录A.LINUX 基础】了解相关背景知识。

1.下载并安装虚拟机软件

VMware Workstation Player：VMware Workstation Player - VMware Customer Connect

2.下载Ubuntu镜像

建议下载Ubuntu18.04/ubuntu-18.04.5-desktop-amd64.iso：Index of /ubuntu-releases/18.04/ (zju.edu.cn)

3.通过虚拟机软件安装Ubuntu镜像

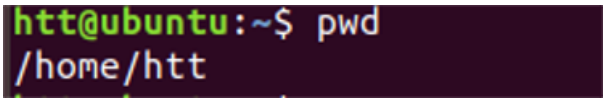
自行搜索相应教程进行安装，建议将用户名设置为姓名缩写，**注意将硬盘大小设置到30G以上方便后续实验。**

4.自行学习Linux命令行的使用，了解基本命令

推荐资源：

- 课程概览与 shell · the missing semester of your cs education (missing-semester-cn.github.io)（这门课程的其他小节也建议大家学习一下）
- TLCL (billie66.github.io)（建议阅读第二~五章、第十二章、十三章相关内容）

请依照第一行的例子，解释以下命令并附上运行截图（推荐截图工具：Snipaste），适当缩放图片大小。

命令	作用	截图
pwd	打印出当前工作目录的名称	
ls		
ls -al		
cd ~		
mkdir oslab		
vi test.c		
gedit test.c		
rm test.c		
sudo apt install curl		
touch a.txt		
cat a.txt | tail -n 10		

3.2 安装 Docker 环境并创建容器(20%)

请参考【附录B.Docker使用基础】了解相关背景知识。

通常情况下，`$` 提示符表示当前运行的用户为普通用户，`#` 代表当前运行的用户为特权用户。但在下文的示例中，以 `###` 开头的行代表注释，`$` 开头的行代表在虚拟机上运行的命令，`#` 开头的行代表在 `docker` 中运行的命令，`(gdb)` 开头的行代表在 `gdb` 中运行的命令。

在执行每一条命令前，请你对将要进行的操作进行思考，给出的命令不需要全部执行，并且不是所有的命令都可以无条件执行，请不要直接复制粘贴命令去执行。

1.安装docker

```
### 使用官方安装脚本自动安装docker
$ curl -fsSL https://get.docker.com | bash -s docker --mirror Aliyun

### 将用户加入docker组，免 sudo
$ sudo usermod -aG docker $USER   ### 注销后重新登陆生效
```

【请将该步骤中命令的输入及运行结果截图附在此处，示例如下。接下来每一步同理，不作赘述，】

因为之前在windows 安装了docker, 所以不再重复安装.

2.下载并导入docker镜像

为便于开展实验，我们在docker镜像中提前安装好了实验所需的环境（RISC-V工具链、QEMU模拟器），并设置好了 `$PATH` 变量以直接访问 `qemu-system-riscv64` 等程序。请下载该docker镜像文件并将其传至虚拟机中（可以直接拖拽到文件夹中），输入命令导入镜像。

```
### 首先进入oslab.tar所在的文件夹，然后使用该命令导入docker镜像
$ cat oslab.tar | docker import - oslab:2020
### 执行命令后若出现以下错误提示
### ERROR: Got permission denied while trying to connect to the Docker daemon
socket at unix:///var/run/docker.sock
### 可以使用下面命令为该文件添加权限来解决
### $ sudo chmod a+rw /var/run/docker.sock

### 查看docker镜像
$ docker image ls
REPOSITORY          TAG                 IMAGE ID            CREATED
SIZE
oslab               2020                d7046ea68221        5 seconds ago
2.89G
```

我下载了oslab.tar , 然后导入到之前安装的windows docker中

```
docker import C:\Users\12638\Downloads\oslab.tar oslab:2020
```

```
C:\Users\12638\Downloads>docker images
REPOSITORY                                          TAG                         IMAGE ID       CREATED
 SIZE
<none>                                              <none>                      fa20c284c064   About a minute ago
 2.89GB
tensorflow1.13.1                                    1.0                         8906e510de1a   5 weeks ago
 1.96GB
1263810658/pytorch1.8.1withgit.cuda10.2.cudnn7.runtime   latest                      5da714cfe33c   8 weeks ago
 4.57GB
pytorch/pytorch                                     1.8.1-cuda10.2-cudnn7-runtime   8feef0e83aed   3 months ago
 4.3GB
tensorflow/tensorflow                               latest                      1d932048a281   4 months ago
 1.3GB
ubuntu                                              latest                      7e0aa2d69a15   4 months ago
 72.7MB
mcr.microsoft.com/mssql/server                      2019-latest                 62c72d863950   5 months ago
 1.49GB
```

上图可以看出导入成功 ,但是没有tag. 用`docker tag`命名一下.

3.从镜像中创建一个容器并进入该容器

```
### 从镜像创建一个容器
$ docker run -it oslab:2020 /bin/bash    ### -i:交互式操作 -t:终端
root@368c4cc44221:/#                     ### 提示符变为 '#' 表明成功进入容器 后面的字符
串根据容器而生成，为容器id
root@368c4cc44221:/# exit (或者CTRL+D)    ### 从容器中退出 此时运行docker ps，运行容器
的列表为空

### 查看当前运行的容器
$ docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED
STATUS              PORTS               NAMES
### 查看所有存在的容器
$ docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED
STATUS                      PORTS               NAMES
368c4cc44221        oslab:2020          "/bin/bash"         54 seconds ago
Exited (0) 30 seconds ago                       relaxed_agnesi

### 启动处于停止状态的容器
$ docker start 368c     ### 368c 为容器id的前四位，id开头的几位便可标识一个容器
$ docker ps             ### 可看到容器已经启动
CONTAINER ID        IMAGE               COMMAND             CREATED
STATUS              PORTS               NAMES
```

```
368c4cc44221        oslab:2020          "/bin/bash"         About a minute ago
Up 16 seconds                           relaxed_agnesi

### 进入已经运行的容器 oslab的密码为2020
$ docker exec -it -u oslab -w /home/oslab 36 /bin/bash
oslab@368c4cc44221:~$
```

我创建容器,按上述操作了一遍.

```
docker run -it ljy/oslab:v1 /bin/bash
docker start cf0e
docker exec -it -u oslab -w /home/oslab cf /bin/bash
```

```
C:\Users\12638\Downloads>docker run -it ljy/oslab:v1 /bin/bash
root@cf0eecc1eaa2:/# exit
exit

C:\Users\12638\Downloads>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

C:\Users\12638\Downloads>docker ps -a
CONTAINER ID   IMAGE                                          COMMAND                  CREATED
            PORTS     NAMES
cf0eecc1eaa2   ljy/oslab:v1                                   "/bin/bash"              2 minutes ago
inute ago             charming_ptolemy
1bfb3ad33d38   tensorflow1.13.1:1.0                           "/bin/bash"              5 weeks ago
go                    tf1
5eeb778924e5   pytorch/pytorch:1.8.1-cuda10.2-cudnn7-runtime  "/bin/bash"              7 weeks ago
 ago                  friendly_haibt
2bcf531522df   mcr.microsoft.com/mssql/server:2019-latest     "/opt/mssql/bin/perm…"   5 months ago
ago                   sql1

C:\Users\12638\Downloads>docker start cf0e
cf0e

C:\Users\12638\Downloads>docker ps
CONTAINER ID   IMAGE          COMMAND       CREATED         STATUS         PORTS     NAMES
cf0eecc1eaa2   ljy/oslab:v1   "/bin/bash"   2 minutes ago   Up 6 seconds             charming_ptolemy

C:\Users\12638\Downloads>docker exec -it -u oslab -w /home/oslab 36 /bin/bash
Error: No such container: 36

C:\Users\12638\Downloads>docker exec -it -u oslab -w /home/oslab 36 /bin/bash
                         docker exec -it -u oslab -w /home/oslab cf /bin/bash
oslab@cf0eecc1eaa2:~$
```

可以看到都成功了.

请解释这两行命令:

- `docker run -it oslab:2020 /bin/bash`

  答: `docker run` 指创建一个新的容器并运行一个命令, `-it` 指交互模式, `oslab:2020` 指定了所使用的镜像, `/bin/bash` 指在容器内执行 `/bin/bash` 命令。

- `docker exec -it -u oslab -w /home/oslab 36 /bin/bash`

  答: `docker exec` 指Run a command in a running container. `-it` 指交互模式, -u 指定了username, -w 指定了working directory. 36是container 的id, 我这里是cf, /bin/bash是运行bash.

3.3 编译 linux 内核(20%)

请参考【附录E.LINUX 内核编译基础】了解相关背景知识。

再次强调, 以 `###` 开头的行代表注释, `$` 开头的行代表在虚拟机上运行的命令, `#` **开头的行代表在 `docker` 中运行的命令, 请勿将 `#` 一并复制入命令行中执行。**

```
### 进入实验目录并设置环境变量
# pwd
/home/oslab
# cd lab0
# export TOP=`pwd`
# mkdir -p build/linux
# make -C linux O=$TOP/build/linux \
          CROSS_COMPILE=riscv64-unknown-linux-gnu- \
          ARCH=riscv CONFIG_DEBUG_INFO=y \
          defconfig all -j$(nproc)
### 编译内核需要一定时间，请耐心等待。
```

编译了大约十分钟

```
  MODINFO modules.builtin.modinfo
  GEN     modules.builtin
  LD      .tmp_vmlinux.kallsyms1
  KSYM    .tmp_vmlinux.kallsyms1.o
  LD      .tmp_vmlinux.kallsyms2
  KSYM    .tmp_vmlinux.kallsyms2.o
  LD      vmlinux
  SYSMAP  System.map
  MODPOST Module.symvers
  OBJCOPY arch/riscv/boot/Image
  CC [M]  fs/nfs/flexfilelayout/nfs_layout_flexfiles.mod.o
  GZIP    arch/riscv/boot/Image.gz
  LD [M]  fs/nfs/flexfilelayout/nfs_layout_flexfiles.ko
  Kernel: arch/riscv/boot/Image.gz is ready
make[1]: Leaving directory '/home/oslab/lab0/build/linux'
make: Leaving directory '/home/oslab/lab0/linux'
oslab@cf0eecc1eaa2:~/lab0$
```

3.4 使用QEMU运行内核(20%)

请参考【附录C.QEMU使用基础】了解相关背景知识。

注意，QEMU的退出方式较为特殊，需要先按住`ctrl+a`，放开后再按一次`x`。

```
### 用户名root，没有密码
# qemu-system-riscv64 -nographic -machine virt -kernel
build/linux/arch/riscv/boot/Image  \
 -device virtio-blk-device,drive=hd0 -append "root=/dev/vda ro console=ttyS0"
\
 -bios default -drive file=rootfs.ext4,format=raw,id=hd0 \
 -netdev user,id=net0 -device virtio-net-device,netdev=net0
```

```
oslab@cf0eecc1eaa2:~/lab0$ qemu-system-riscv64 -machine help
Supported machines are:
none                 empty machine
sifive_e             RISC-V Board compatible with SiFive E SDK
sifive_u             RISC-V Board compatible with SiFive U SDK
spike                RISC-V Spike Board (default)
spike_v1.10          RISC-V Spike Board (Privileged ISA v1.10)
spike_v1.9.1         RISC-V Spike Board (Privileged ISA v1.9.1)
virt                 RISC-V VirtIO board
```

```
[    0.816848] usbcore: registered new interface driver usbhid
[    0.820820] usbhid: USB HID core driver
[    0.827870] NET: Registered protocol family 10
[    0.840832] Segment Routing with IPv6
[    0.844477] sit: IPv6, IPv4 and MPLS over IPv4 tunneling dri
[    0.851721] NET: Registered protocol family 17
[    0.855718] 9pnet: Installing 9P2000 support
[    0.858915] Key type dns_resolver registered
[    0.917719] EXT4-fs (vda): mounted filesystem with ordered d
[    0.920506] VFS: Mounted root (ext4 filesystem) readonly on
[    0.926346] devtmpfs: mounted
[    0.993251] Freeing unused kernel memory: 232K
[    0.999762] Run /sbin/init as init process
[    1.245018] EXT4-fs (vda): re-mounted. Opts: (null)
[    1.247036] ext4 filesystem being remounted at / supports ti
Starting syslogd: OK
Starting klogd: OK
Running sysctl: OK
Starting mdev... OK
modprobe: can't change directory to '/lib/modules': No such fil
Initializing random number generator: OK
Saving random seed: [    8.208387] random: dd: uninitialized ur
OK
Starting network: udhcpc: started, v1.31.1
udhcpc: sending discover
udhcpc: sending select for 10.0.2.15
udhcpc: lease of 10.0.2.15 obtained, lease time 86400
deleting routers
adding dns 10.0.2.3
OK

Welcome to Buildroot
buildroot login: root
# ls
#
```

登录成功

3.5 使用 gdb 调试内核(20%)

请参考【附录D.GDB使用基础】了解相关背景知识。学会调试将在后续实验中为你提供帮助，推荐同学们跟随GDB调试入门指南教程完成相应基础练习，熟悉gdb调试的使用。

```
### Terminal 1
# qemu-system-riscv64 -nographic -machine virt -kernel
build/linux/arch/riscv/boot/Image  \
 -device virtio-blk-device,drive=hd0 -append "root=/dev/vda ro console=ttyS0"
\
 -bios default -drive file=rootfs.ext4,format=raw,id=hd0 \
 -netdev user,id=net0 -device virtio-net-device,netdev=net0 -S -s
```

再打开一个终端窗口，从宿主机进入该容器，并切换到`lab0`文件夹，进入gdb调试。

```
###Terminal2
# riscv64-unknown-linux-gnu-gdb build/linux/vmlinux
```

如下图, 在第二个terminal 进入gdb调试

```
build  linux  rootfs.ext4
oslab@cf0eecc1eaa2:~/lab0$ riscv64-unknown-linux-gnu-gdb build/linux/vmlinux
GNU gdb (GDB) 9.1
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=x86_64-pc-linux-gnu --target=riscv64-unknown-linux-g
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/linux/vmlinux...
(gdb)
```

顺序执行下列gdb命令，写出每条命令的含义并附上执行结果的截图。

```
(gdb) target remote localhost:1234
```

- 含义：target remote 命令表示远程调试，而1234是默认的用于调试连接的端口号。
- 执行结果：

```
For help, type "help".
Type "apropos word" to search for commands relat
Reading symbols from build/linux/vmlinux...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
0x0000000000001000 in ?? ()
```

```
(gdb) b start_kernel
(gdb) b *0x80000000
(gdb) b *0x80200000
(gdb) info breakpoints
(gdb) delete 2
(gdb) info breakpoints
```

- 含义： 设置三个断点 , 查看已设置断点信息, 删除第二个断点,再次查看已设置断点信息
- 执行结果：

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/linux/vmlinux...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
0x0000000000001000 in ?? ()
(gdb) b start_kernel
Breakpoint 1 at 0xffffffe000001714: file /home/oslab/lab0/linux/init/main.c, line 837.
(gdb) b *0x80000000
Breakpoint 2 at 0x80000000
(gdb) b *0x80200000
Breakpoint 3 at 0x80200000
(gdb) info breakpoints
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0xffffffe000001714 in start_kernel at /home/oslab/lab0/linux/init/main.c:837
2       breakpoint     keep y   0x0000000080000000
3       breakpoint     keep y   0x0000000080200000
(gdb) delete 2
(gdb) info breakpoints
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0xffffffe000001714 in start_kernel at /home/oslab/lab0/linux/init/main.c:837
3       breakpoint     keep y   0x0000000080200000
(gdb)
```

```
(gdb) continue #继续执行程序，直到再次遇到断点处：
(gdb) delete 3
(gdb) continue
(gdb) step #如果我们想跟踪函数内部的情况，可以使用step命令（可简写为s），它可以单步跟踪到函数内部，但前提是该函数有调试信息并且有源码信息。
(gdb) s #step的简写
(gdb) (不做输入，直接回车) # 重复上一次命令s
(gdb) next #用于在程序断住后，继续执行下一条语句，假设已经启动调试，并在第12行停住，如果要继续执行，则使用n执行下一条语句
(gdb) n #next的简写.
(gdb) (不做输入，直接回车)# 重复上一次命令n
```

- 含义：写在注释中
- 执行结果：

```
(gdb) continue
Continuing.

Breakpoint 3, 0x0000000080200000 in ?? ()
(gdb) delete 3
(gdb) continue
Continuing.

Breakpoint 1, start_kernel () at /home/oslab/lab0/linux/init/main.c:837
837             set_task_stack_end_magic(&init_task);
(gdb) step
set_task_stack_end_magic (tsk=<optimized out>) at /home/oslab/lab0/linux/kernel/fork.c:863
863             *stackend = STACK_END_MAGIC;    /* for overflow detection */
(gdb) s
start_kernel () at /home/oslab/lab0/linux/init/main.c:838
838             smp_setup_processor_id();
(gdb)
smp_setup_processor_id () at /home/oslab/lab0/linux/arch/riscv/kernel/smp.c:38
38              cpuid_to_hartid_map(0) = boot_cpu_hartid;
(gdb) next
start_kernel () at /home/oslab/lab0/linux/init/main.c:841
841             cgroup_init_early();
(gdb) n
843             local_irq_disable();
(gdb)
844             early_boot_irqs_disabled = true;
(gdb)
```

```
(gdb) disassemble # 查看汇编代码,不带参数：默认的反汇编范围是所选择地址附近的汇编代码；
(gdb) nexti #每次执行一条机器指令,不进入函数
(gdb) n # next的简写.
(gdb) stepi #每次执行一条机器指令
(gdb) s # step的简写
```

- 含义：如上
- 执行结果：

```
(gdb) disassemble
Dump of assembler code for function start_kernel:
   0xffffffe000001714 <+0>:     addi    sp,sp,-80
   0xffffffe000001716 <+2>:     sd      ra,72(sp)
   0xffffffe000001718 <+4>:     sd      s0,64(sp)
   0xffffffe00000171a <+6>:     sd      s1,56(sp)
   0xffffffe00000171c <+8>:     addi    s0,sp,80
   0xffffffe00000171e <+10>:    sd      s2,48(sp)
   0xffffffe000001720 <+12>:    sd      s3,40(sp)
   0xffffffe000001722 <+14>:    sd      s4,32(sp)
   0xffffffe000001724 <+16>:    sd      s5,24(sp)
   0xffffffe000001726 <+18>:    sd      s6,16(sp)
   0xffffffe000001728 <+20>:    auipc   a0,0x100a
   0xffffffe00000172c <+24>:    addi    a0,a0,1560 # 0xffffffe00100bd40 <init_task>
   0xffffffe000001730 <+28>:    auipc   ra,0x205
   0xffffffe000001734 <+32>:    jalr    92(ra) # 0xffffffe00020678c <set_task_stack_end_magic>
   0xffffffe000001738 <+36>:    jal     ra,0xffffffe000003730 <smp_setup_processor_id>
   0xffffffe00000173c <+40>:    jal     ra,0xffffffe000008d4e <cgroup_init_early>
   0xffffffe000001740 <+44>:    csrci   sstatus,2
=> 0xffffffe000001744 <+48>:    li      a5,1
   0xffffffe000001746 <+50>:    auipc   a4,0x106f
   0xffffffe00000174a <+54>:    sb      a5,-1786(a4) # 0xffffffe00107004c <early_boot_irqs_disabled>
   0xffffffe00000174e <+58>:    jal     ra,0xffffffe000004606 <boot_cpu_init>
   0xffffffe000001752 <+62>:    auipc   a1,0x9ff
   0xffffffe000001756 <+66>:    addi    a1,a1,-1682 # 0xffffffe000a000c0 <linux_banner>
   0xffffffe00000175a <+70>:    auipc   a0,0xb3d
   0xffffffe00000175e <+74>:    addi    a0,a0,-850 # 0xffffffe000b3e408
   0xffffffe000001762 <+78>:    auipc   ra,0x245
   0xffffffe000001766 <+82>:    jalr    -510(ra) # 0xffffffe000246564 <printk>
   0xffffffe00000176a <+86>:    addi    a0,s0,-72
   0xffffffe00000176e <+90>:    auipc   s4,0x106f
   0xffffffe000001772 <+94>:    addi    s4,s4,-1798 # 0xffffffe001070068 <initrd_end>
   0xffffffe000001776 <+98>:    jal     ra,0xffffffe0000033b4 <setup_arch>
```

```
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
(gdb) nexti
0xffffffe000001746      844             early_boot_irqs_disabled = true;
(gdb) n
850             boot_cpu_init();
(gdb) stepi
boot_cpu_init () at /home/oslab/lab0/linux/arch/riscv/include/asm/current.h:31
31              return riscv_current_is_tp;
(gdb) s
2492            set_cpu_online(cpu, true);
(gdb)
```

- **请回答：nexti和next的区别在哪里？stepi和step的区别在哪里？next和step的区别是什么？**

  答： next是源代码的下一行,不进入函数, nexti是 单步机器指令,不进入函数

  step是单步到下一个不同的源代码行, 包括进入函数, stepi是单步机器指令. next 不会进入到函数内部, step会进入到函数内部

```
(gdb) continue #继续执行程序，直到再次遇到断点处：
(gdb) quit  #退出gdb
```

- 含义： 如上
- 执行结果：

```
(gdb) n
850             boot_cpu_init();
(gdb) stepi
boot_cpu_init () at /home/oslab/lab0/linux/arch/riscv/include/asm/current.h:31
31              return riscv_current_is_tp;
(gdb) s
2492            set_cpu_online(cpu, true);
(gdb) continue
Continuing.
q
```

没有断点了, 机器就启动了 .

```
Program received signal SIGINT, Interrupt.
arch_cpu_idle () at /home/oslab/lab0/linux/arch/riscv/kernel/process.c:3
33                local_irq_enable();
(gdb) continue
Continuing.
^C
Program received signal SIGINT, Interrupt.
arch_cpu_idle () at /home/oslab/lab0/linux/arch/riscv/kernel/process.c:3
33                local_irq_enable();
(gdb) quit
A debugging session is active.

        Inferior 1 [process 1] will be detached.

Quit anyway? (y or n) y
Detaching from program: /home/oslab/lab0/build/linux/vmlinux, process 1
Ending remote debugging.
[Inferior 1 (process 1) detached]
oslab@cf0eecc1eaa2:~/lab0$
```

4 讨论和心得

请在此处填写实验过程中遇到的问题及相应的解决方式。由于本实验为新实验，可能存在不足之处，欢迎同学们对本实验提出建议。

- 3.2.2 出现问题 “<”运算符是为将来使用而保留的。

`docker load < C:\Users\12638\Downloads\oslab.tar` 谷歌一下, 因为powershell不支持, 尝试用cmd可以解决.

- 3.2.2出现问题open /var/lib/docker/tmp/docker-import-673032073/bin/json: no such file or directory**因为压缩包如果是用 docker save 打包的，就可以用 docker load，但是如果压缩包是用 docker export 打包的，那就需要用 docker import**

`cat oslab.tar | docker import - ljy/oslab:2021 # 这是不行的cmd没有cat`

过程中查阅了https://docs.docker.com/engine/reference/commandline/import/

- 在disassemble这里直接输入nexti似乎没用.

```
   0xffffffe00000177a <+102>:   ld      s3,0(s4)
   0xffffffe00000177e <+106>:   auipc   s2,0x106f
--Type <RET> for more, q to quit, c to continue without paging--nexti
   0xffffffe000001782 <+110>:   addi    s2,s2,-1806 # 0xffffffe001070070 <initrd_star
   0xffffffe000001786 <+114>:   beqz    s3,0xffffffe0000017d4 <start_kernel+192>
   0xffffffe00000178a <+118>:   addi    s1,s3,-12
   0xffffffe00000178e <+122>:   li      a2,12
   0xffffffe000001790 <+124>:   auipc   a1,0xb3d
   0xffffffe000001794 <+128>:   addi    a1,a1,-896 # 0xffffffe000b3e410
   0xffffffe000001798 <+132>:   mv      a0,s1
   0xffffffe00000179a <+134>:   auipc   ra,0x47f
```

我就先q退出了, 再输入nexti.

- continue后, 程序就一直运行.

可以看到terminal1 一开始是阻塞的, `gdb continue` 后可以登录了. 为了让程序停下来，我们可以发送一个中断信号（ctrl + c），GDB捕捉到该信号后会挂起被调试进程。

```
(gdb) continue
Continuing.
q
quit

^C
Program received signal SIGINT, Interrupt.
arch_cpu_idle () at /home/oslab/lab0/linux/arch/riscv/kernel/process.c:33
33              local_irq_enable();
```

这个时候 terminal1也不能输入了.

再次continue后, 会把之前terminal1 的输入给显示出来.

```
Saving random seed: [    7.915366] random: dd: uninitia
OK
Starting network: udhcpc: started, v1.31.1
udhcpc: sending discover
udhcpc: sending select for 10.0.2.15
udhcpc: lease of 10.0.2.15 obtained, lease time 86400
deleting routers
adding dns 10.0.2.3
OK

Welcome to Buildroot
buildroot login: root
# lsssllllss2lllslllslllslslslllslssssss
```